

中国科学院大学计算机组成原理实验报告

姓名	周禹驰	学号	2024K8009929027
专业	计算机科学与技术	实验编号	Prj4-选作实验 I
实验名称	功能型处理器结构优化:RISC-V32 流水线处理器		

完整实验流程包括设计、开发、调试三个步骤:

- **设计步骤**为在代码编写前的准备阶段,主要包括模块划分、流水级边界确定、冒险处理方案和分支预测结构设计,并绘制设计图纸。
- **开发步骤**为在 S-IDE 集成开发环境中进行 Verilog 代码编写,实现五级流水线、流水寄存器、前递单元、阻塞冲刷控制和性能计数器。
- **调试步骤**包括本地仿真验证后推送至云平台进行 FPGA 原型验证,以及根据 simple、medium、advanced 和性能测试结果进行迭代修正。

一、设计成果(对设计流程的总结。需附上设计结构图,可用 PPT 或者在线绘图软件绘制逻辑结构框图后保存为 PDF,再插入到 L^AT_EX 中)

1 总体设计目标

本实验基于原有多周期 RISC-V32 处理器,按照讲义中“功能型处理器结构优化”的要求,将处理器改造为顺序单发射五级流水线。设计重点不是扩展指令集,而是在保持原译码方法和提交接口正确的前提下,提高指令吞吐率,并通过前递、气泡、冲刷和 gshare 分支预测降低流水线冒险造成的性能损失。

本设计保留三个性能计数器:cycle、retire 和 mem_req。其中 cycle 用于统计总周期,retire 用于统计提交指令数,mem_req 用于统计数据访存请求数,便于计算 CPI 并分析访存负载。

2 五级流水线结构

处理器划分为 IF、ID、EX、MEM、WB 五个流水级。IF 级负责取指请求和分支预测;ID 级负责译码、立即数生成和寄存器堆读取;EX 级负责 ALU、乘法、移位、分支条件和跳转目标计算;MEM 级负责阻塞式数据访存;WB 级负责寄存器写回和 inst_retire 提交信息生成。

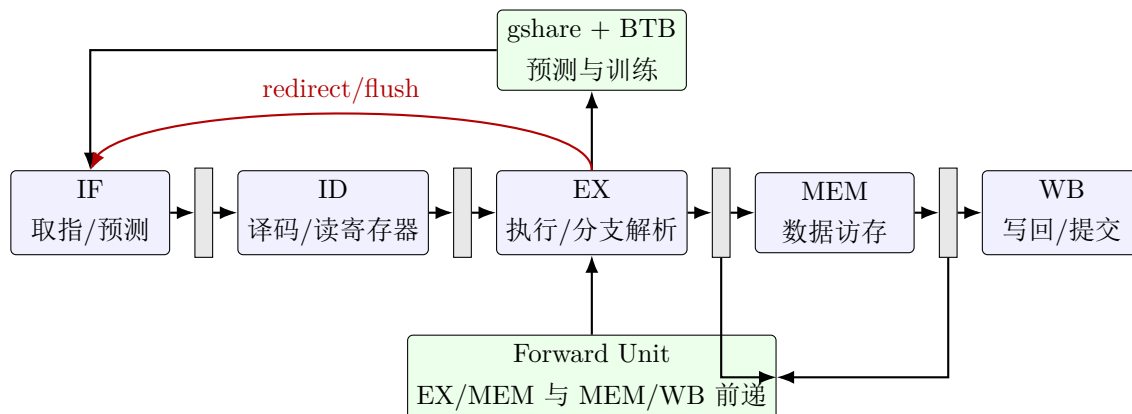


图 1: RISC-V32 五级流水线总体结构

3 流水寄存器与模块划分

按照模块化要求,流水线边界被拆分为 `if_id_reg`、`id_ex_reg`、`ex_mem_reg` 和 `mem_wb_reg` 四类流水寄存器。在具体实现中使用通用 `pipeline_reg.v` 承载宽数据总线,并在顶层对不同级间信号进行打包和解包。这样可以减少顶层中大量离散寄存器造成的混乱,也便于统一实现 `stall`、`flush` 和 `reset` 行为。

4 冒险处理设计

数据冒险主要通过前递单元和 `load-use` 气泡处理解决。对普通 ALU 结果,EX/MEM 阶段结果优先前递;若 EX/MEM 不能提供所需结果,则从 MEM/WB 阶段前递。Load 指令的数据直到 MEM 阶段读响应返回后才有效,因此当 ID 级指令依赖前方未完成 Load 结果时,流水线插入气泡并冻结前端。

冒险类型	处理方法	说明
ALU 到 ALU	前递	EX/MEM 非 Load 结果直接送入 EX 输入
WB 到 EX	前递	MEM/WB 写回结果送入 EX 输入
WB 到 ID	读口旁路	同周期写后读返回最新写回值
Load-use	插入气泡	等待 Load 读数据返回后再继续执行
控制冒险	冲刷与重定向	EX 级解析真实下一 PC 并清除错误路径指令

表 1: 流水线冒险处理策略

5 gshare 分支预测器设计

分支预测器由全局历史寄存器 GHR、两位饱和计数器 PHT 和 BTB 组成。取指时使用 `pc[7:2] xor ghr` 索引 PHT,同时查询 BTB;当 BTB tag 命中且 PHT 高位为 1 时预测跳转,否则预测顺序执行。EX 级得到真实分支方向和目标地址后训练 PHT,并在实际跳转时更新 BTB。

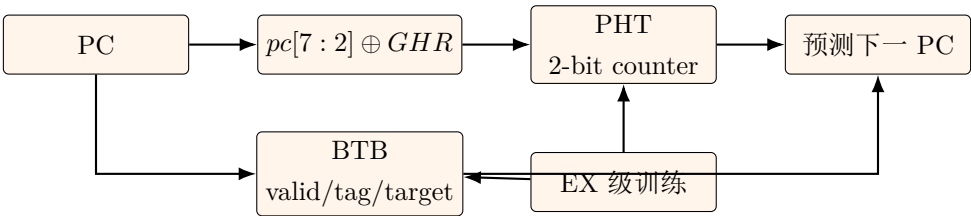


图 2: gshare 分支预测器结构

二、 开发成果(对代码开发流程的总结。需附上包含注释的 Verilog HDL 关键代码段、相应信号的仿真波形和信号变化的说明)

1 顶层流水线控制实现

顶层文件 `custom_cpu.v` 负责组织五级流水线、访存握手、分支重定向和性能计数器。ID 级译码沿用原多周期处理器中的指令分类方式,避免因重新设计译码表导致指令行为偏离原实现。多路选择器尽量采用掩码与或结构,减少连续三目运算造成的可读性下降。

Listing 1: 掩码式下一 PC 选择示例

```
assign pc_next = ({32{pc_from_aluout}} & aluout) |
                 ({32{pc_from_alures}} & exec_result) |
```

```
{32{~(pc_from_aluout | pc_from_alures)}} & pc_seq);
```

在流水线版本中,同样采用类似思路选择预测 PC、重定向 PC、顺序 PC 和前递数据。这样做的优点是每一路来源的使能条件清晰,便于在波形中检查是哪一路控制了最终结果。

2 前递单元实现

前递单元输入 EX 级源寄存器号、EX/MEM 阶段写回信息和 MEM/WB 阶段写回信息,输出两个两位选择信号。EX/MEM 阶段非 Load 结果优先级最高;MEM/WB 阶段作为第二优先级;若均不匹配则选择 ID/EX 寄存器中保存的原始操作数。

Listing 2: 前递优先级逻辑摘要

```
assign forward_a_ex = ex_mem_valid & ex_mem_reg_write &
    ~ex_mem_mem_read & (ex_mem_rd != 5'd0) &
    (ex_mem_rd == id_ex_rs1);
assign forward_a_wb = mem_wb_valid & mem_wb_reg_write &
    (mem_wb_rd != 5'd0) &
    (mem_wb_rd == id_ex_rs1);
```

3 阻塞与冲刷实现

流水线阻塞主要来自两类情况:第一,IF 级或 MEM 级正在等待外部接口响应;第二,ID 级检测到 load-use 相关。冲刷主要来自 EX 级分支解析后发现预测错误。处理顺序上,重定向优先于普通推进;被冲刷的指令不能产生寄存器写回和访存副作用。

关键波形选取了三类最容易引入错误的场景:load-use 相关导致的前端阻塞、EX/MEM 阶段到 EX 操作数的前递,以及分支预测错误后的 flush 与 redirect。这三组信号能够覆盖数据冒险和控制冒险的主要调试路径。

4 调试过程与失败分析

流水线版本的调试先从行为仿真开始,使用 simple、medium 和 advanced 用例检查提交接口。定位错误时不直接从取指端逐拍追踪,而是先找到 inst_retire 第一次与参考模型不一致的位置,再反推该指令在 EX、MEM 和 WB 级的来源选择。这样可以快速区分错误属于译码、前递、访存返回、冲刷控制还是写回选择。

早期失败主要集中在三类路径。第一类是数据相关路径,典型现象是算术、移位或 Load 后继指令提交值错误;原因是 EX 级操作数没有正确选择 EX/MEM 或 MEM/WB 中的最新结果,或者 load-use 场景没有插入气泡。第二类是控制相关路径,典型现象是分支预测错误后仍提交错误路径指令;原因是前端已有取指响应返回,若不丢弃错误路径响应,就会把已经失效的指令重新送入流水线。第三类是访存阻塞路径,典型现象是 MEM 级长时间等待或重复发起请求;原因是 ready/valid 握手跨多拍时,请求完成和读数据完成状态没有被稳定保存。

在通过行为仿真后,FPGA 实现阶段仍出现过失败。流水线处理器相比多周期版本增加了级间寄存器、前递比较网络、冲刷阻塞控制和分支预测结构,组合路径和布线扇出明显增加,导致时序压力变大。云平台运行 custom_bit_gen 时曾出现超时或 FPGA 错误,这类失败不一定表示 RTL 功能错误,更可能发生在综合、布局或布线阶段。处理时需要结合实现日志判断失败位置:若行为仿真已通过而错误出现在 place/route 或 bitstream 生成阶段,则优先从降低关键路径、减少高扇出控制信号、简化前递选择和降低额外调试逻辑入手。

4.1 关键波形分析

??给出了 load-use 相关触发后的阻塞过程。第 984 个周期开始,ID 级指令的 id_rs2 为 x14,而 EX/MEM 阶段的 Load 指令目的寄存器 ex_mem_rd 同样为 x14,因此 load_use_mem 和 id_load_use_stall 被拉高。由

EX/MEM forwarding to EX rs2, cycles 971-976



At cycle 973, EX rs2=x13 matches EX/MEM rd=x13; operand2 is forwarded as 0x4c.

图 4: EX/MEM 阶段结果前递到 EX 级操作数

branch misprediction flush and redirect, cycles 43-52

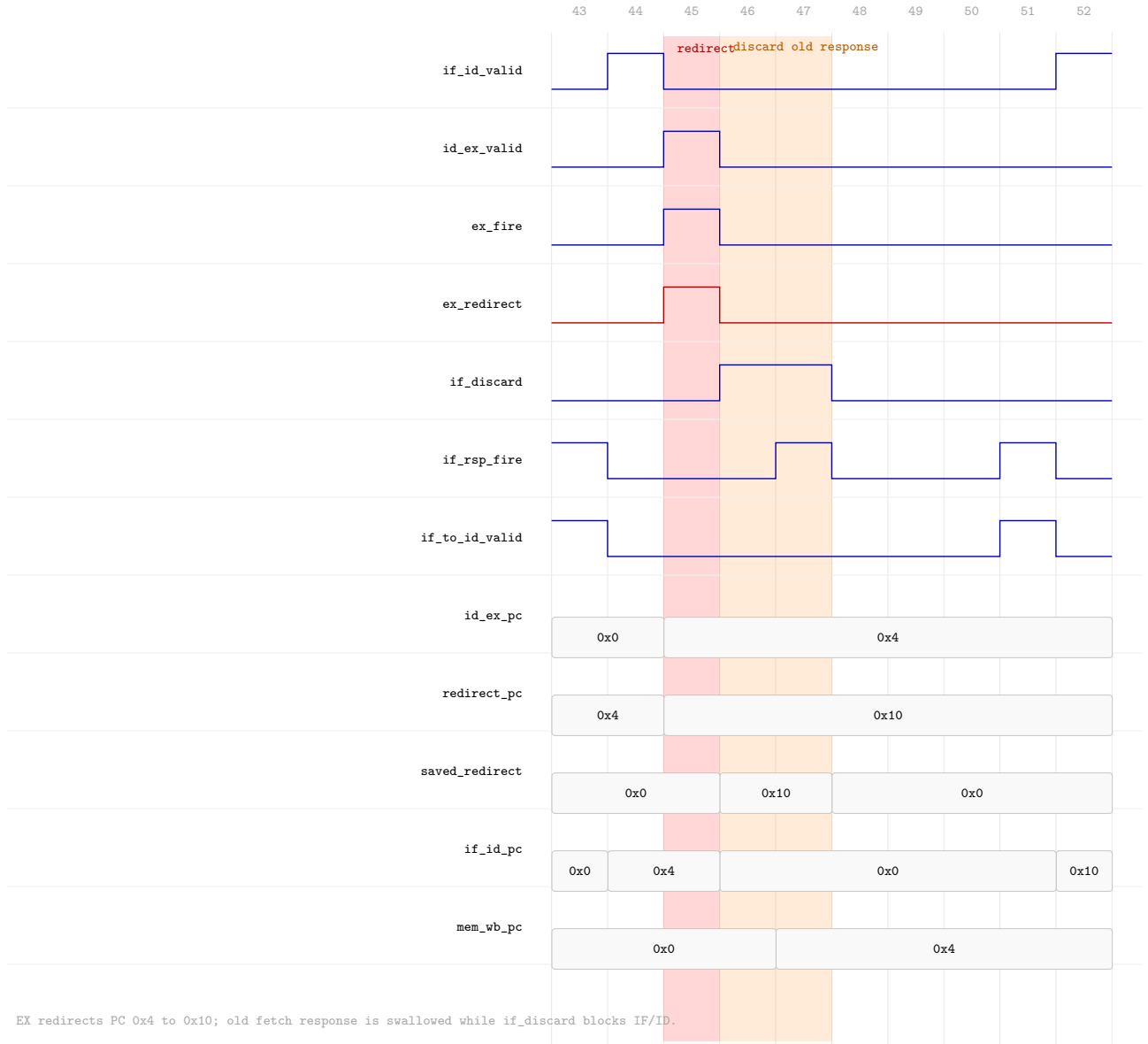


图 5: 分支预测错误后的 flush、redirect 与旧取指响应丢弃

程序	多周期 cycles	流水线 cycles	周期加速比	多周期时间/ms	流水线时间/ms	时间加速比
15pz	532,389,755	24,159,010	22.04	5337.89	169.94	31.41
bf	39,877,154	1,934,574	20.61	702.34	29.96	23.44
dinic	1,474,612	179,303	8.22	6599.38	202.53	32.58
fib	181,088,840	10,310,248	17.56	1825.54	77.09	23.68
md5	385,527	25,628	15.04	200.50	12.94	15.49
qsort	783,929	41,253	19.00	955.10	35.22	27.12
queen	6,885,054	338,571	20.34	81.16	10.43	7.78
sieve	745,218	43,234	17.24	20.26	9.53	2.13
ssort	44,731,346	2,584,444	17.31	1345.51	52.54	25.61
合计	808,361,435	39,616,265	20.40	17067.68	600.18	28.44

表 3: 多周期 CPU 与五级流水线 CPU 的 microbench 性能对比

不同程序的收益存在差异。15pz、bf、qsort、queen 等程序的周期加速比接近或超过 20 倍,说明其指令流能够较充分地利用流水线重叠执行; dinic 的周期加速比为 8.22 倍,相对较低,说明该程序更容易受到访存等待、分支冲刷或外部接口响应的限制。sieve 的周期加速比仍有 17.24 倍,但日志时间加速比只有 2.13 倍,说明 FPGA 端到端时间还受到程序装载、测试框架、外设交互等非 CPU 核心周期因素影响,因此报告中同时列出 cycles 和实测时间,避免只用单一指标判断性能。

流水线版本 microbench 总计提交指令数为 3,389,060,总周期为 39,616,265,整体 CPI 约为 11.69。该 CPI 仍明显高于理想流水线的 1,主要原因是当前设计为顺序单发射阻塞流水线: I-cache 取指等待、D-cache 或 DDR 访存等待、load-use 气泡以及分支预测错误都会阻止流水线持续提交。也就是说,流水线结构主要降低了多周期 CPU 中每条指令固定经历多个阶段的开销,但外部存储和控制冒险仍然是后续优化空间。

三、 学习总结(总结设计、开发与调试过程中遇到的问题,及实验中的收获与不足)

1 问题总结(实验过程中遇到的问题、对问题的思考过程及解决方法)

序号	问题类别	出错模块	问题现象	产生原因	解决办法
1	逻辑电路设计	EX/MEM 前递	shift 等 advanced 用例写回值错误	移位/ALU 结果相关未在 EX 级取得最新操作数	调整前递优先级,确保 EX/MEM 非 Load 结果优先
2	行为仿真	分支冲刷	PC 进入错误路径后提交错误指令	IF 级已有未完成取指请求,redirect 后响应仍可能返回	增加错误路径响应丢弃逻辑,并在 EX 级解析后冲刷前端
3	FPGA 调试	MEM 阻塞	程序长时间不提交或超时	数据访存 ready/-valid 握手多拍状态未保存完整	使用请求完成和读数据完成标志保存 MEM 阶段进度
4	FPGA 实现	custom_bit_gen	bitstream 生成超时或失败	流水线控制、前递网络和预测器增加时序与布线压力	结合实现日志定位失败阶段,优先简化关键路径和高扇出控制
5	上板验证	FPGA 运行	FPGA 测试报错或结果不稳定	行为仿真未覆盖的接口等待、错误路径响应或时序边界问题	对照 retire 日志和波形回溯,分别排查功能错误与实现时序错误

表 4: 流水线处理器问题总结

- **流水线处理器问题补充。**流水线调试的关键是不要只看单条指令的组合逻辑,而要观察指令在五个级之间的生命周期。尤其是分支预测错误、Load 返回延迟和 Store 写数据相关这三类情况,容易出现“前一拍看起来正确,后一拍提交错误”的问题。

对于 FPGA 阶段的失败,需要先区分功能错误和实现错误。若 simple、medium 或 advanced 行为仿真已经稳定通过,但 custom_bit_gen 在综合、布局布线或生成 bitstream 阶段超时,则更可能是时序压力或布线拥塞导致;若 bitstream 生成成功但上板运行报错,则应回到 retire 提交日志和关键波形,检查访存握手、flush 优先级和错误路径响应处理是否仍存在边界场景。

- **其他关键模块问题补充。**性能计数器的定义必须保持稳定,否则软件侧跑分输出会失去比较意义。本设计最终只保留 cycle、retire 和 mem_req 三个计数器,其余位置置零。

2 收获总结(在数字电路设计方法、RTL 编写方法、数字电路验证方法及其他方面的收获)

类别	名称	体会	后续计划
数字电路设计	流水线划分	级间信号必须围绕提交正确性组织,不能只按组合逻辑自然切分	继续优化 IF 和 MEM 阻塞带来的 CPI 损失
RTL 编写	模块化寄存器	统一流水寄存器能显著降低顶层寄存器数量和 flush/stall 错误概率	进一步细化不同级间寄存器的命名和注释
验证方法	retire 回溯	从错误提交点反推流水级状态,比直接从取指端排查更高效	建立固定的波形检查清单

表 5: 收获总结

四、 扩展思考(对讲义中思考题(如有)的理解和回答)

- **思考题 1:为什么流水线处理器仍然可能比理想 CPI=1 慢很多?**

理想 CPI=1 假设每个周期都能提交一条指令,但真实处理器会受到取指等待、数据访存等待、load-use 气泡、分支预测错误冲刷和外设访问延迟影响。对于本设计,I-cache 和 D-cache 未命中会直接冻结相关流水级;分支在 EX 级才解析,因此一次预测错误至少损失前端若干拍。

- **思考题 2:为什么采用 gshare 而不是一位预测?**

一位预测只能记录单个分支最近一次方向,遇到循环退出、交替分支或受上下文影响的分支时容易反复误判。gshare 利用全局历史与 PC 异或索引 PHT,可以在较小表项规模下区分不同路径上下文,通常比单纯 PC 索引的局部预测更适合小型顺序流水线。

五、 实验时长

在课后,你花费了大约 50 小时完成此次实验,其中各个阶段的用时如表??所示。

步骤	时长(小时)	说明
设计	8	五级流水级划分、冒险处理和 gshare 预测器设计
开发	30	RTL 实现、模块拆分和性能计数器整理
调试	10	行为仿真、FPGA 评测和性能测试迭代

表 6: 实验时长